

LABORATORY OBSERVATION

Student Name: D.vijaya sri

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5-2

1. TITLE

Node.js File Management

2. AIM

To create a Node.js application that accepts a filename and content from the user and performs file creation/writing, reading, appending, and final content display using the Node.js File System module.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the Node.js File System (fs) module.
 - To create and write data to a file.
 - To read existing file contents.
 - To append additional content to a file.
 - To display the final contents after file operations.
 - To handle invalid input and runtime errors.
-

4. THEORY

Node.js provides built-in modules for performing server-side operations. The File System (fs) module provides methods for creating, reading, writing, and modifying files.

This application uses `writeFileSync()` to create a file and write the initial content, `readFileSync()` to read the file, and `appendFileSync()` to add additional content to the existing file.

The `readline` module is used to accept filename and content from the user through the terminal. A Promise-based helper function is used so that the input operations can be performed sequentially.

5. ALGORITHM / PROCEDURE

1. Import the File System (fs) and readline modules.
2. Create a readline interface for terminal input.
3. Ask the user to enter a filename.
4. Ask the user to enter the initial content.
5. Validate that the filename is not empty.
6. Create the file and write the initial content using `writeFileSync()`.

7. Read and display the contents using `readFileSync()`.
 8. Ask the user for additional content.
 9. Append the additional content using `appendFileSync()`.
 10. Read and display the final file contents.
 11. Handle errors using try-catch and close the readline interface.
-

6. PROGRAM

app.js

```
const fs = require("fs");

const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function askQuestion(question) {
  return new Promise((resolve) => {
    rl.question(question, resolve);
  });
}

async function fileManagement() {
  try {
    const fileName = (await askQuestion("Enter filename: ")).trim();

    const initialContent =
      await askQuestion("Enter content to write: ");

    if (!fileName) {
      console.log("Error: Filename cannot be empty.");
      rl.close();
      return;
    }

    // Create/write the file
    fs.writeFileSync(fileName, initialContent);

    console.log(
      "\nFile created and content written successfully."
    );

    // Read the file contents

    const contentAfterWrite =
```

package.json

```
{
  "name": "q2-nodejs-file-management",
  "version": "1.0.0",
  "description": "Node.js file management using the File System module",
  "main": "app.js",
```

7. CODE EXPLANATION

Code	Explanation
require("fs")	Imports the built-in Node.js File System module.
require("readline")	Imports the module used to accept input from the terminal.
readline.createInterface()	Creates an interface for reading user input.
askQuestion()	Returns user input as a Promise so operations can run sequentially.
writeFileSync()	Creates the file and writes the initial content.
readFileSync()	Reads and returns the file contents.
appendFileSync()	Adds additional content to the existing file.
try...catch	Handles errors that may occur during file operations.
rl.close()	Closes the readline interface after execution.

8. TEST CASES AND EXECUTION DATA

Test Case	Input	Expected Result	Actual Result	Status
TC-01	sampleFile; the file has created; file details	File is created, read, appended, and final content is displayed.	File created, read, appended, and final content displayed.	Pass
TC-02	Empty filename	An appropriate filename error is displayed.	Not provided	—
TC-03	Additional content: file details	Additional content is appended to the existing file.	file details appended successfully.	Pass
TC-04	File operation error	The error should be caught and displayed.	Not provided	—

9. OUTPUT

Output 1: Terminal Execution of Node.js File Management

```
C:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 5\Q2_NodeJS_File_Management\src>node app.js
Enter filename: samplefile
Enter content to write: the file has created

File created and content written successfully.

File contents after writing:
the file has created

Enter additional content to append: file details

Additional content appended successfully.

Final file contents:
the file has created file details

C:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 5\Q2_NodeJS_File_Management\src>
```

The supplied screenshot shows successful file creation and writing, reading of the initial contents, appending of additional content, and display of the final contents.

10. RESULT

Thus, the Node.js File Management application was implemented using the File System module. The supplied execution output verifies successful file creation, writing, reading, appending, and display of the final file contents for the demonstrated test.

How this would be submitted

Source Code

```
Q2_NodeJS_File_Management/
├── src/
│   └── app.js
├── package.json
└── README.md
```